

3.5.3 Repetition Statements (for loop, while loop)

لوپ (Loop) کوڈ کے ایک بلاک کو بار بار چلاتا ہے۔

CS XI

Chapter 3: Programming Fundamentals

Lecture 11 of 13

★ Golden

3.5.3 Repetition Statements (for loop, while loop) ★ Golden

LEARN IT

A **repetition statement (loop)** lets a block of code run **again and again**. This greatly reduces the need to write the same code many times. Each single run of the loop body is called an **iteration**.

1. The for loop is used when the **exact number of iterations is known**. It takes each value from a sequence one by one and stores it in the **loop variable** (e.g. `i`).

The `range()` function generates the sequence of numbers for the loop:

Syntax	How it works	Example	Values produced
<code>range(stop)</code>	From 0 up to, but not including , stop.	<code>range(3)</code>	0 1 2
<code>range(start, stop)</code>	From start up to, but not including, stop.	<code>range(2, 5)</code>	2 3 4
<code>range(start, stop, step)</code>	From start towards stop, jumping by step.	<code>range(1, 6, 2)</code>	1 3 5

```
for i in range(3):
    print(i)
```

OUTPUT

```
0
1
2
```

```
for i in range(1, 6, 2):
    print(i)
```

OUTPUT

```
1
3
5
```

```
for i in range(2, 5):
    print(i)
```

```
for i in range(0, 6, 2):
    print("Even:", i)
```

OUTPUT

2
3
4

OUTPUT

Even: 0
Even: 2
Even: 4

You can see all the values of a range at once by converting it to a list. A negative step counts **down**; if start equals stop, the range is empty:

```
print(list(range(5)))  
print(list(range(2, 5)))  
print(list(range(1, 6, 2)))  
print(list(range(10, 0, -2)))  
print(list(range(5, 5)))
```

OUTPUT

```
[0, 1, 2, 3, 4]  
[2, 3, 4]  
[1, 3, 5]  
[10, 8, 6, 4, 2]  
[]
```

WORKED EXAMPLE

Multiplication table of 7:

```
for i in range(1, 6):  
    print(f"7 x {i} = {7 * i}")
```

OUTPUT

```
7 x 1 = 7  
7 x 2 = 14  
7 x 3 = 21  
7 x 4 = 28  
7 x 5 = 35
```

Adding numbers with an accumulator. An **accumulator** is a variable that starts at 0 and collects a running total.

```
total = 0  
for i in range(1, 11):  
    total += i  
print("Sum of 1 to 10 =", total)
```

OUTPUT

```
Sum of 1 to 10 = 55
```

i	1	2	3	4	5	...	10
total after	1	3	6	10	15	...	55

Looping over a string or list. A `for` loop can also take items directly from any sequence:

```
for ch in "Sindh":  
    print(ch, end=" ")  
print()  
for m in [85, 90, 75]:  
    print(m + 5)
```

OUTPUT

```
S i n d h  
90  
95  
80
```

REMEMBER IT

range stops one short: `range(1, 11)` gives 1 to **10**. To include a number `n`, write `range(..., n + 1)`. Number of values in `range(a, b)` = `b - a`.

COMMON MISTAKE

Writing `range(1, 10)` to print 1 to 10 misses 10 — an "off-by-one" logical error. Also, the loop body must be indented; a `for` line without an indented block gives `IndentationError: expected an indented block after 'for' statement.`

اردو میں سمجھیں

لوپ (Loop) کوڈ کے ایک بلاک کو بار بار چلاتا ہے۔ `for` لوپ تب استعمال ہوتا ہے جب پہلے سے معلوم ہو کہ کتنی بار دہرانا ہے۔ `range(stop)` صفر سے شروع ہوتا ہے، `start`، `range(start, stop)` سے، اور `range(start, stop, step)` ہر بار `step` کے برابر چھلانگ لگاتا ہے۔ آخری عدد (`stop`) کبھی شامل نہیں ہوتا۔

EXAM TIP

Expect "predict the output" questions on `range()` with a step, and programs such as "print a table", "sum of first n numbers", "even numbers from 1 to 20". Always state what range produces before tracing.

CHECK YOURSELF

1. Predict the output:

```
for i in range(10, 0, -3):
    print(i, end=" ")
```

2. Predict the output:

```
s = 0
for k in range(2, 11, 2):
    s += k
print(s)
```

3. Find the error:

```
for i in range(3):
    print(i)
```

Answers: 1. 10 7 4 1. 2. 30 (2 + 4 + 6 + 8 + 10). 3. The body is not indented: `IndentationError`. Indent `print(i)` by 4 spaces.

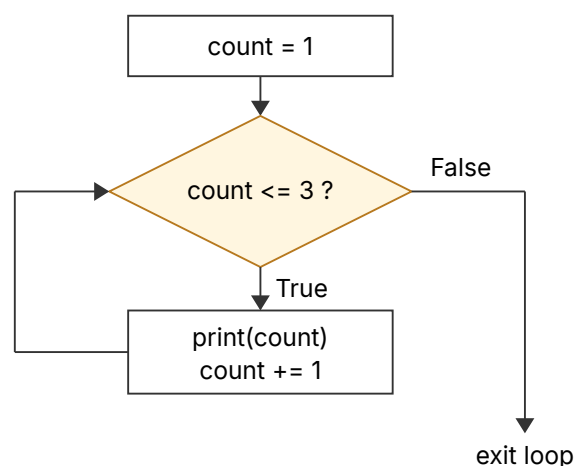
LEARN IT

2. The `while` loop repeats a block of code **as long as a condition remains True**. It is used when the exact number of iterations is **not known in advance** (for example, "keep asking until the password is correct").

A `while` loop needs three parts: (1) **initialise** a variable before the loop, (2) **test** the condition, (3) **update** the variable inside the loop. If you forget the update, the condition never becomes False and the loop never stops — an **infinite loop**.

Loop control statements change the normal flow of a loop:

- `break` — exits (stops) the loop **entirely and immediately**.
- `continue` — **skips the rest of the current iteration** and jumps to the next one.



A while loop: test the condition, run the body, go back and test again.

```
count = 1
while count ≤ 3:
    print(count)
    count += 1    # VERY IMPORTANT: update the condition!
```

OUTPUT

```
1
2
3
```

WORKED EXAMPLE

Trace of the while loop above:

count	count <= 3?	Printed	count after update
1	True	1	2
2	True	2	3
3	True	3	4
4	False	—	loop ends

Sum of digits — we do not know how many digits the number has, so `while` fits well:

```
n = 4072
total = 0
while n > 0:
    digit = n % 10
    total += digit
    n = n // 10
print("Sum of digits =", total)
```

OUTPUT

```
Sum of digits = 13
```

Digits taken: 2, 7, 0, 4 → 2 + 7 + 0 + 4 = 13. `n` becomes 407, 40, 4, 0, and then the loop stops.

Password check — repeat until the user gets it right:

```
password = input("Enter password: ")
while password != "sindh123":
    print("Wrong password, try again.")
    password = input("Enter password: ")
print("Welcome!")
```

Sample run (user types `abc`, `sindh`, `sindh123`):

OUTPUT

```
Enter password: abc
Wrong password, try again.
Enter password: sindh
Wrong password, try again.
Enter password: sindh123
Welcome!
```

break and continue side by side:

```
for i in range(5):
    if i == 3:
        break
    print(i)
print("Loop ended")
```

```
OUTPUT
0
1
2
Loop ended
```

At $i = 3$ the loop stops completely, so 3 and 4 are never printed.

```
for i in range(5):
    if i == 3:
        continue
    print(i)
```

```
OUTPUT
0
1
2
4
```

At $i = 3$ only that iteration is skipped; the loop carries on with 4.

`while True` **with** `break` — a loop that runs "forever" until `break` stops it:

```
n = 1
while True:
    if n * n > 50:
        break
    n += 1
print("First number whose square is more than 50:", n)
```

```
OUTPUT
First number whose square is more than 50: 8
```

REMEMBER IT

break = "exit the room"; **continue** = "skip this turn". For every `while` loop, check the **I-T-U** rule: Initialise, Test, Update.

COMMON MISTAKE

Infinite loop — find the error:

```
count = 1
while count ≤ 5:
    print(count)
```

`count` is never updated, so `count ≤ 5` stays True and the program prints 1 forever. Fix: add `count += 1` inside the loop. (To stop a running program in the terminal, press **Ctrl + C**.)

`while` لوپ اس وقت تک چلتا رہتا ہے جب تک شرط درست (True) رہے، اور یہ تب استعمال ہوتا ہے جب پہلے سے معلوم نہ ہو کہ کتنی بار دہرانا ہے۔ لوپ کے اندر ویری ایبل کو اپڈیٹ کرنا لازمی ہے ورنہ لوپ کبھی ختم نہیں ہوگا (Infinite Loop)۔ `break` لوپ کو فوراً مکمل طور پر روک دیتا ہے، جبکہ `continue` صرف موجودہ چکر چھوڑ کر اگلے چکر پر چلا جاتا ہے۔

EXAM TIP

"Differentiate `break` and `continue` with examples" and "What is an infinite loop?" are frequent. Use the two side-by-side programs above with their outputs — examiners like seeing the difference in the output (0 1 2 vs 0 1 2 4).

CHECK YOURSELF

1. Predict the output:

```
n = 1
while n < 20:
    n = n * 3
print(n)
```

2. Predict the output:

```
for i in range(1, 8):
    if i % 3 == 0:
        continue
    if i == 7:
        break
    print(i, end=" ")
```

Answers: 1. 27 (n: 1 → 3 → 9 → 27; 27 < 20 is False, so the loop stops). 2. 1 2 4 5 (3 and 6 are skipped by `continue`; at 7 the loop breaks).

LEARN IT

A **nested loop** is a loop inside another loop. The **inner loop runs completely, from start to finish, for every single iteration of the outer loop**. If the outer loop runs 2 times and the inner loop runs 3 times, the inner body runs $2 \times 3 = 6$ times.

```
for outer in range(1, 3):          # runs 2 times (1, 2)
    for inner in range(1, 4):      # runs 3 times per outer (1, 2, 3)
        print(f"Outer: {outer}, Inner: {inner}")
```

OUTPUT

```
Outer: 1, Inner: 1
Outer: 1, Inner: 2
Outer: 1, Inner: 3
Outer: 2, Inner: 1
Outer: 2, Inner: 2
Outer: 2, Inner: 3
```

WORKED EXAMPLE

A star triangle. The outer loop chooses the row; the inner loop prints that many stars on the same line (`end=""`), and the empty `print()` moves to the next line.

```
for row in range(1, 5):
    for col in range(row):
        print("*", end="")
    print()
```

OUTPUT

```
*
**
***
****
```

Row 1: inner loop runs once → *. Row 2: twice → **. And so on up to row 4.

for loop vs while loop

for loop	while loop
Used when the number of iterations is known .	Used when the number of iterations is unknown (condition-based).
Automatically updates the loop variable.	Requires a manual update of the counter (risk of an infinite loop).
Cleaner and shorter for counting tasks.	Better for dynamic situations (like "keep playing until the user says Stop").
<code>for i in range(1, 4): print(i)</code>	<code>i = 1 / while i ≤ 3: / print(i) / i += 1</code>

REMEMBER IT

Ask yourself: "Do I know **how many times**?" Yes → `for`. "Do I only know **when to stop**?" → `while`. Nested loops are like a clock: the minute hand (inner loop) goes all the way round for each step of the hour hand (outer loop).

COMMON MISTAKE

Indentation decides which loop a line belongs to. In the triangle program, if `print()` were indented under the inner loop, every star would go on its own line.

اردو میں سمجھیں

نیسٹڈ لوپ (Nested Loop) میں ایک لوپ دوسرے لوپ کے اندر ہوتا ہے، اور باہر والے لوپ کے ہر چکر میں اندر والا لوپ شروع سے آخر تک مکمل چلتا ہے۔ جب دہرانے کی تعداد معلوم ہو تو `for` لوپ، اور جب صرف رکنے کی شرط معلوم ہو تو `while` لوپ بہتر ہے۔

EXAM TIP

"Differentiate between for and while loops" (give three points and an example of each) and "How many times will the inner statement run?" (multiply the counts) are typical. For pattern programs, explain what the outer and inner loops each control.

CHECK YOURSELF

1. Predict the output:

```
count = 0
for i in range(3):
    for j in range(2):
        count += 1
print(count)
```

2. Which loop would you use to keep reading numbers until the user enters 0?

Answers: 1. 6 (3×2). 2. A `while` loop, because the number of inputs is not known in advance.

LEARN IT

This program combines all three control structures: **sequence** (steps in order), **selection** (checking "done", checking the price, checking for a discount) and **repetition** (a `while` loop that keeps adding items). It uses a **flag** — a Boolean variable (`shopping`) that controls when the loop stops. `item.lower()` converts the text to lowercase so that "done", "Done" and "DONE" all work.

```

print("===== SHOPPING BILL =====")
total = 0.0
count = 0
shopping = True

# REPETITION: keep adding items
while shopping:
    item = input("Enter Item Name (or 'done' to stop): ")
    # SELECTION: check if user wants to stop
    if item.lower() == "done":
        shopping = False
    else:
        price = float(input(f"Enter Price of {item}: "))
        # SELECTION: validate price
        if price <= 0:
            print("Invalid price!")
        else:
            total += price
            count += 1

# Summary and discount
if total >= 5000:
    discount = total * 0.20 # 20% off
else:
    discount = 0

final = total - discount
print(f"Total Items: {count}")
print(f"Total Bill: {total}")
print(f"Final to Pay: {final}")

```

WORKED EXAMPLE

Sample run. The user buys Rice for 3000, types a wrong price -50 for Soap, buys Oil for 2500, then types DONE :

OUTPUT

```
===== SHOPPING BILL =====  
Enter Item Name (or 'done' to stop): Rice  
Enter Price of Rice: 3000  
Enter Item Name (or 'done' to stop): Soap  
Enter Price of Soap: -50  
Invalid price!  
Enter Item Name (or 'done' to stop): Oil  
Enter Price of Oil: 2500  
Enter Item Name (or 'done' to stop): DONE  
Total Items: 2  
Total Bill: 5500.0  
Final to Pay: 4400.0
```

Step	item	price	total	count
1	Rice	3000.0	3000.0	1
2	Soap	-50.0 (invalid, not added)	3000.0	1
3	Oil	2500.0	5500.0	2
4	DONE → shopping = False	—	5500.0	2

After the loop: $5500.0 \geq 5000$, so discount = $5500.0 \times 0.20 = 1100.0$, and final = $5500.0 - 1100.0 = \mathbf{4400.0}$.

REMEMBER IT

Label the parts of any program with comments, as here: `# REPETITION`, `# SELECTION`. It helps you, your teacher and the examiner see the structure.

COMMON MISTAKE

`total` and `count` must be set to 0 **before** the loop. If you put `total = 0.0` inside the loop, it resets on every item and the bill is wrong.

یہ شاپنگ بل پروگرام تینوں کنٹرول اسٹرکچرز کو ملا کر بنایا گیا ہے: `while` لوپ سے اشیاء بار بار شامل ہوتی ہیں، `if-else` سے غلط قیمت روکی جاتی ہے اور 5000 یا زیادہ کے بل پر 20 فیصد رعایت دی جاتی ہے۔

CHECK YOURSELF

1. What will "Final to Pay" be if the user buys one item for 4000 and then types `done` ?
2. What is the purpose of the variable `shopping` ?

Answers: 1. Final to Pay: 4000.0 (4000 is less than 5000, so no discount). 2. It is a flag that keeps the `while` loop running; setting it to `False` stops the loop.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. for i in range(3) runs:

- a. i=0,1,2
- b. i=1,2,3
- c. i=3,2,1
- d. forever

MCQ 2. while repeats while the condition is:

- a. True
- b. False
- c. A file
- d. None always

MCQ 3. break:

- a. Exits the loop immediately
- b. Skips one line only
- c. Closes Python
- d. Commits SQL

MCQ 4. continue:

- a. Skips the rest of this iteration
- b. Ends the program
- c. Defines a function
- d. Opens a file

MCQ 5. A nested loop: inner runs:

- a. Fully for each outer step
- b. Once in total
- c. Never
- d. Only if NAND

MCQ 6. range(2, 6) yields:

- a. 2,3,4,5
- b. 2,3,4,5,6
- c. 1,2,3,4,5,6
- d. 6,5,4

Extra short questions

1. When for vs while?
2. What is an infinite loop?
3. Trace s=0; for i in range(1,4): s+=i.
4. Shopping bill uses which loop typically?

Extra long question

1. Write nested loops to print a 3×3 table of i,j. Then rewrite the shopping-bill outline with a while, a running total, and an if on quantity.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. for: known count / iterate a sequence. while: unknown count, stop on a condition.
2. while True without break, or a condition that never becomes False.
3. i=1 s=1; i=2 s=3; i=3 s=6.
4. while more items (unknown count) or for a known list of items.

Long

1. for i in range(3): for j in range(3): print(i,j). Bill: total=0; while True: read item/qty; if qty<=0: continue; if item=="end": break; total+=...; print total.

Next lecture: 3.6 Libraries in Python (Built-in & Third-Party)